

Documento de Diseño - Entrega 2

Proyecto: Alineamiento de Secuencias de ADN
Programación Paralela y Concurrente | Junio 2026

1. Objetivo del rediseño

La segunda entrega optimiza la solución del primer entregable incorporando una versión Pthreads con concurrencia más eficiente y una implementación OpenCL para ejecutar la búsqueda exacta de patrones en GPU o CPU compatible. La versión secuencial se mantiene como referencia de correctitud.

2. Arquitectura general

El flujo conserva una única generación de datos para comparar todas las estrategias con la misma secuencia de ADN y el mismo conjunto de patrones.

Etapa	Responsabilidad	Salida
Entrada y parámetros	Leer longitud de ADN, cantidad de patrones, longitud de patrón y número de hilos.	Estructura parametros_t.
Generación de datos	Crear secuencia de ADN y patrones aleatorios con los nucleótidos A, T, C y G.	cadena_adn y arreglo patron_t.
Referencia secuencial	Ejecutar fuerza bruta de un patrón contra todas las posiciones.	Primera posición encontrada o NO_ENCONTRADO.
Pthreads optimizado	Repartir dinámicamente patrones entre un pool persistente de hilos.	Resultados equivalentes a la versión secuencial.
OpenCL	Ejecutar work-items por patrón y posición; conservar la primera coincidencia.	Resultados copiados desde buffers OpenCL.
MPI	Distribuir patrones entre procesos y recolectar resultados en el proceso root.	Resultados globales para impresión final.

3. Módulos principales

Módulo	Función principal
cadena.c / cadena.h	Reserva memoria, genera ADN, genera patrones y define patron_t.

parametros.c / parametros.h	Procesa argumentos de línea de comandos y valores por defecto.
busqueda_lineal.c	Implementa la búsqueda secuencial base por fuerza bruta.
busqueda_pthread.c	Implementa el pool persistente de hilos y asignación dinámica de patrones.
busqueda_opencl.c / .cl	Prepara buffers, compila el kernel y ejecuta la búsqueda paralela OpenCL.
busqueda_MPI.c	Distribuye patrones entre procesos MPI y recolecta resultados.
tests/test.c	Valida casos determinísticos y compara versiones contra la secuencial.

4. Optimización con Pthreads

La versión Pthreads optimizada aplica tres mejoras pedidas por el enunciado:

- Pool de hilos persistente: los hilos se crean una vez y permanecen activos hasta llamar a destruir_hilos.
- Asignación dinámica: cada hilo toma el siguiente patrón disponible mediante un índice compartido.
- Sincronización con condiciones: pthread_cond_wait y pthread_cond_broadcast evitan espera activa cuando no hay trabajo.

Las condiciones de carrera se evitan protegiendo el índice siguiente_patron, el contador trabajos_pendientes y la bandera activo con un pthread_mutex. Cada hilo escribe únicamente en la posición del patrón que tomó, por lo que no existe conflicto entre resultados de patrones distintos.

5. Diseño de la solución OpenCL

La implementación OpenCL se divide entre código host en C y un kernel escrito en OpenCL C. En el host, los patrones se aplanan en un arreglo contiguo y se copian junto con la secuencia de ADN, longitudes y resultados iniciales hacia buffers del dispositivo.

Componente	Descripción
Selección de dispositivo	Se intenta usar GPU y, si no existe, CPU compatible con OpenCL.
Buffers	ADN, patrones aplanados, longitudes y resultados se crean con clCreateBuffer.
Kernel 2D	La dimensión 0 representa el patrón y la dimensión 1 representa la posición

	de la secuencia.
Coincidencia	Cada work-item compara un patrón contra una posición específica.
Primera posición	atomic_min actualiza el resultado del patrón conservando la menor posición encontrada.
Retorno a CPU	Los resultados se leen con clEnqueueReadBuffer y se traducen a estado COINCIDENCIA o NO_ENCONTRADO.

6. Validación y casos de prueba

La prueba automatizada compara las versiones paralelas contra la búsqueda secuencial, que funciona como referencia. OpenCL se valida cuando el equipo tiene plataforma/dispositivo disponible; si no, la prueba OpenCL se omite sin invalidar las demás pruebas.

Caso	Secuencia / configuración	Patrón	Resultado esperado
1	ATGCGT	ATG	Posición 0
2	ATGCGT	CGT	Posición 3
3	ATGCGT	AAA	No encontrado
4	AAAAAA	AAA	Posición 0, primera coincidencia
5	ATGCGT	ATGCGT	Posición 0, patrón igual a secuencia
6	ATG	ATGC	No encontrado, patrón más largo que secuencia
7	N=100000, K=50, L=5	Aleatorios	Mismos resultados que la versión secuencial

7. Comparación de rendimiento

Los datos de esta tabla quedan pendientes para completar manualmente después de ejecutar el programa en el equipo donde se evaluará la entrega. Se recomienda correr cada configuración al menos tres veces y anotar el promedio.

Versión	Configuración	Tiempo (s)	Patrones encontrados
---------	---------------	------------	----------------------

Secuencial	N=100000, K=100, P=5	0.000238	100/100
Pthreads optimizado	Hilos=8	0.000465	100/100
OpenCL	Dispositivo=GPU	0.224354	100/100
MPI	Procesos=8	0.316183	100/100

9. Comandos sugeridos

- Compilar: make
- Ejecutar pruebas: make test
- Ejecutar programa: make run
- Ejecutar MPI con 4 procesos: make run_mpi
- Ejemplo de parámetros: ./bin/programa --longitud-adn 100000 --cantidad-patron 50 --longitud-patron 5 --numero-hilos 4